202401753 Bily Vallecidos Folgar
202404421 Jose Andres Reyes
202400527 Adrian Gilberto Recinos
202404597 Allan Marcelo Perez
202405037 Diego Obed Patzan
202303371 Cristian Saban Sequen

TALLER #4

MANUAL TECNICO
GRUPO #7

USAC – FACULTAD DE CIENCIAS Y SISTEMAS

Introducción

El presente manual técnico tiene como finalidad documentar el desarrollo de la aplicación web realizada como parte de la práctica de la asignatura. A lo largo del proyecto se aplicaron diferentes tecnologías y herramientas modernas que permitieron construir una solución funcional, orientada a resolver una necesidad real dentro de la Facultad de Ingeniería.

El proceso de desarrollo representó una experiencia valiosa, ya que permitió poner en práctica conocimientos sobre programación cliente-servidor, manejo de bases de datos y control de versiones en un entorno colaborativo. Asimismo, trabajar en equipo dentro de un repositorio en línea fortaleció las habilidades de organización y comunicación, elementos clave en el ámbito profesional.

De esta manera, este manual servirá como guía técnica que describe la arquitectura, las funciones implementadas en el servidor, así como los aspectos principales que garantizan el correcto funcionamiento de la aplicación.

Objetivos

Objetivo General

Documentar de manera clara y detallada el funcionamiento técnico de la aplicación web desarrollada, asegurando que pueda comprenderse, mantenerse y escalarse en un futuro.

Objetivo Específico

Facilitar el uso y mantenimiento de la aplicación a través de lineamientos técnicos bien definidos.

Especificaciones Técnicas

Hardware

- Procesador: Intel Core i3 de 2.0 GHz o equivalente (se recomienda Core i5 o superior).
- Memoria RAM: 4 GB mínimo (8 GB recomendados para un mejor rendimiento en VS Code y ejecución simultánea de backend y frontend).
- Espacio en Disco: 200 MB para instalación de Node.js y VS Code.
- 500 MB adicionales para dependencias y librerías de Node.js.
- 200 MB para dependencias del frontend (React + Vite).
- Disco duro: 2 GB libres como mínimo.

- Pantalla: Resolución mínima de 1280x720 (se recomienda 1366x768 o superior).
- Periféricos: Teclado y mouse (recomendado).

Software

- Sistema Operativo:
    1. Windows 10/11 (64 bits recomendado).
    2. GNU/Linux (Ubuntu 20.04 en adelante).
    3. macOS 10.13 (High Sierra) en adelante.
- Lenguaje de Programación:
    1. JavaScript (ECMAScript 6 o superior).
    2. JSX (para el desarrollo con React).
- Entorno de ejecución:
    1. Node.js versión 18 o superior (incluye npm versión 9 o superior).
- Editor de Código:
    1. Visual Studio Code versión 1.70 o superior.
- Frameworks y librerías utilizadas:

  Backend:
    1. Express (para creación de la API REST).
    2. Nodemon (para reinicio automático del servidor en desarrollo).
    3. MySQL2 (para conexión con la base de datos).

  Frontend:

    1. React (para el desarrollo de la interfaz de usuario).
    2. Vite (como entorno de construcción y servidor de desarrollo).
    3. @vitejs/plugin-react (para integración de React con Vite).

  Base de Datos:

    1. MySQL versión 8.0 (local o en la nube de docker hub).

Lógica de la Aplicación

Paquetes Utilizados

Nombre del Paquete	Descripción	Objetos dentro del Paquete
Backend	Paquete en el que se encuentra toda la lógica para el funcionamiento de la aplicación, en donde hacemos las autenticaciones, definimos rutas, definimos la plantilla para nuestra base datos para luego solo hacer una inyección de contenido a la base de datos que está en la nube de docker y definimos las instrucciones que debe seguir el programa cuando algún usuario inicie sesión, haga alguna operación, etc.	Carpeta src: Carpeta middleware: • Auth,js Carpeta routes: • auth.routes.js • posts.routes.js • users.routes.js Carpeta útil: • validator.js db.js server.js .env Init.sql package-lock.json package.json
Frontend	Paquete que gestiona toda la interfaz de usuario (UI). Contiene el archivo principal (main) y diferentes componentes de React.js que facilitan la construcción de la UI. Desde aquí se realizan las peticiones al backend para mostrar datos dinámicos y responder a las acciones del usuario.	Carpeta src: Carpeta: Components • Filters.jsx/css • PostCard.jsx/.css Carpeta pages • Auth.css • CreatePost.jsx • ForgotPassword.jsx • Home.css • Login.jsx • Profile.jsx • Register.jsx Api.js App.jsx Main.jsx Index.css index.html package-lock.json package.json vite.config.js

Backend

Clases Utilizadas

Middleware/auth.js

El middleware requireAuth se encarga de verificar la autenticación de los usuarios mediante tokens JWT antes de permitir el acceso a rutas protegidas. Este lee el encabezado Authorization de la petición y extrae el token si está presente y comienza con Bearer . Si no se encuentra un token o este es inválido, devuelve un error 401 indicando que el token es requerido o inválido. En caso de que el token sea válido, se extrae la información del usuario, como su id y registro, y se almacena en req.user, permitiendo que la petición continúe hacia la ruta o middleware siguiente mediante next(). Su objetivo es asegurar que solo usuarios autenticados puedan acceder a ciertas funcionalidades de la aplicación, centralizando la gestión de la verificación de tokens y los errores de autenticación.

```
const jwt = require('jsonwebtoken');

function requireAuth(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'Token requerido' });

  try {
    const data = jwt.verify(token, process.env.JWT_SECRET);
    req.user = { id: data.id, registro: data.registro };
    next();
  } catch {
    res.status(401).json({ error: 'Token inválido' });
  }
}
module.exports = { requireAuth };
```

routes/auth.routes.js

Este módulo de rutas de Express maneja la autenticación y gestión de usuarios mediante tres endpoints principales: /register, /login y /reset. Utiliza bcryptjs para hashear y comparar contraseñas, jsonwebtoken (JWT) para generar tokens de autenticación y un pool de base de datos para interactuar con MySQL. En el registro (/register), se reciben y limpian los datos del usuario, se valida que no estén vacíos, que el email sea correcto y que la contraseña tenga al menos 6 caracteres, y se verifica que el usuario no exista; luego se hashea la contraseña y se inserta el nuevo usuario en la base de datos, devolviendo un mensaje de éxito.

En el login (/login), se obtiene el registro y contraseña del usuario, se busca en la base de datos y se compara la contraseña con el hash guardado; si es correcta, se genera un token JWT con el id y registro del usuario, válido por 8 horas, que se devuelve al cliente.

En el reinicio de contraseña (/reset), se validan el registro y email proporcionados, se genera un hash de la nueva contraseña y se actualiza en la base de datos, enviando un mensaje de confirmación. En todos los casos, se manejan errores mediante códigos HTTP apropiados (400, 401, 409, 500) y mensajes descriptivos, asegurando que solo se permita el acceso y las modificaciones a usuarios autenticados y válidos.

```
const { Router } = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const { pool } = require('../db');
const { str, isEmail } = require('../util/validators');

const r = Router();

r.post('/register', async (req, res) => {
  try {
    const registro = str(req.body.registro);
    const nombres  = str(req.body.nombres);
    const apellidos= str(req.body.apellidos);
    const email    = str(req.body.email).toLowerCase();
    const password = str(req.body.password);

    if (!registro || !nombres || !apellidos || !isEmail(email) || password.length
< 6)
      return res.status(400).json({ error: 'Datos inválidos' });

    const [exists] = await pool.query('SELECT id FROM users WHERE registro=? OR
email=?',[registro,email]);
    if (exists.length) return res.status(409).json({ error: 'Usuario ya existe'
});

    const hash = await bcrypt.hash(password, 10);
    await pool.query(
      'INSERT INTO users (registro,nombres,apellidos,email,password_hash) VALUES
(?,?,?,?,?)',
      [registro,nombres,apellidos,email,hash]
    );
    res.status(201).json({ message: 'Registrado' });
  } catch (e) { res.status(500).json({ error: e.message }); }
});
```

```
r.post('/login', async (req, res) => {
  try {
    const registro = str(req.body.registro);
    const password = str(req.body.password);

    const [rows] = await pool.query('SELECT id,registro,password_hash FROM users
WHERE registro=?',[registro]);
    if (!rows.length) return res.status(401).json({ error: 'Credenciales
inválidas' });

    const ok = await bcrypt.compare(password, rows[0].password_hash);
    if (!ok) return res.status(401).json({ error: 'Credenciales inválidas' });

    const token = jwt.sign({ id: rows[0].id, registro: rows[0].registro },
process.env.JWT_SECRET, { expiresIn: '8h' });
    res.json({ token });
  } catch (e) { res.status(500).json({ error: e.message }); }
});

r.post('/reset', async (req, res) => {
  try {
    const registro = str(req.body.registro);
    const email    = str(req.body.email).toLowerCase();
    const newPass  = str(req.body.newPassword);

    const [rows] = await pool.query('SELECT id FROM users WHERE registro=? AND
email=?',[registro,email]);
    if (!rows.length) return res.status(400).json({ error: 'Datos incorrectos'
});
    const hash = await bcrypt.hash(newPass, 10);
    await pool.query('UPDATE users SET password_hash=? WHERE id=?',[hash,
rows[0].id]);
    res.json({ message: 'Contraseña actualizada' });
  } catch (e) { res.status(500).json({ error: e.message }); }
});

module.exports = r;
```

routes/posts.routes.js

Este módulo de rutas de Express maneja la creación y consulta de publicaciones y comentarios en la aplicación. Primero, se importa el Router de Express para definir rutas modulares, la conexión a la base de datos mediante pool, el middleware requireAuth para proteger rutas que requieren usuarios autenticados y la función str para

limpiar y validar entradas. La ruta GET / permite listar todas las publicaciones, aplicando filtros opcionales por curso (courseId), nombre de catedrático (profName) o nombre del curso (courseName), construyendo la consulta de manera segura con parámetros y devolviendo los resultados en formato JSON ordenados por fecha de creación. La ruta POST / permite a usuarios autenticados crear nuevas publicaciones; primero limpia y valida el mensaje, el curso y el nombre del catedrático, asegurándose de que el mensaje no esté vacío y que se especifique un curso o catedrático; luego inserta la publicación en la base de datos usando el id del usuario autenticado.

La ruta POST /:id/comments permite agregar comentarios a publicaciones existentes; limpia y valida el contenido del comentario, verifica que no esté vacío y lo inserta en la base de datos asociándolo al post y al usuario que comenta. En todas las rutas se manejan errores con códigos HTTP adecuados: 400 para datos inválidos, 401 para usuarios no autenticados, 500 para errores internos del servidor, y se devuelven mensajes claros en formato JSON. De esta manera, el módulo asegura que solo usuarios autorizados puedan crear contenido, que los datos sean válidos y que la información se obtenga o almacene de forma segura y consistente.

```
const { Router } = require('express');
const { pool } = require('../db');
const { requireAuth } = require('../middleware/auth');
const { str } = require('../util/validators');

const r = Router();

r.get('/', async (req, res) => {
  try {
    const { courseId, profName, courseName } = req.query;
    const params = [];
    let where = 'WHERE 1=1';
    if (courseId)   { where += ' AND p.course_id=?';
params.push(Number(courseId)); }
    if (profName)   { where += ' AND p.prof_name LIKE ?';
params.push('%'+str(profName)+'%'); }
    if (courseName) { where += ' AND c.name LIKE ?';
params.push('%'+str(courseName)+'%'); }

    const [rows] = await pool.query(
      `SELECT p.id,p.message,p.created_at,
              u.registro,u.nombres,u.apellidos,
              c.id as course_id,c.name as course_name,
              p.prof_name
       FROM posts p
       LEFT JOIN users u ON u.id=p.author_id
       LEFT JOIN courses c ON c.id=p.course_id
```

```
      ${where}
      ORDER BY p.created_at DESC`,
     params
    );
    res.json(rows);
  } catch (e) { res.status(500).json({ error: e.message }); }
});

r.post('/', requireAuth, async (req,res)=>{
  try{
    const message = str(req.body.message);
    const courseId = req.body.courseId ? Number(req.body.courseId) : null;
    const profName = str(req.body.profName) || null;

    if (!message || (!courseId && !profName))
      return res.status(400).json({ error: 'Debe elegir curso o catedrático y
escribir un mensaje' });

    await pool.query(
      'INSERT INTO posts (author_id,course_id,prof_name,message) VALUES
(?,?,?,?)',
      [req.user.id, courseId, profName, message]
    );
    res.status(201).json({ message: 'Publicación creada' });
  }catch(e){ res.status(500).json({ error: e.message }); }
});

r.post('/:id/comments', requireAuth, async (req,res)=>{
  try{
    const content = str(req.body.content);
    if (!content) return res.status(400).json({ error: 'Comentario vacío' });
    await pool.query(
      'INSERT INTO comments (post_id,author_id,content) VALUES (?,?,?)',
      [Number(req.params.id), req.user.id, content]
    );
    res.status(201).json({ message: 'Comentario agregado' });
  }catch(e){ res.status(500).json({ error: e.message }); }
});

module.exports = r;
```

routes/users.routes.js

Este módulo de rutas de Express se encarga de gestionar información de usuarios y cursos. Primero, se importa el Router de Express para definir rutas modulares, la conexión a la base de datos mediante pool, el middleware requireAuth para proteger rutas que requieren autenticación y las funciones str e isEmail para limpiar y validar entradas.

La ruta GET /users/:registro obtiene los datos de un usuario por su registro, consultando la base de datos y devolviendo un error 404 si no existe.

La ruta PUT /me permite a un usuario autenticado actualizar su perfil; primero limpia y valida los datos recibidos (nombres, apellidos y correo), verifica que sean correctos y luego actualiza la información en la base de datos, devolviendo un mensaje de confirmación.

La ruta GET /courses devuelve la lista de todos los cursos disponibles ordenados alfabéticamente, mientras que GET /users/:registro/courses consulta los cursos que un usuario ha aprobado, relacionando las tablas de usuarios, cursos y cursos aprobados.

Finalmente, la ruta POST /me/courses permite a un usuario autenticado agregar un curso aprobado; valida que se envíe un courseId y lo inserta en la base de datos si no existía previamente, devolviendo un mensaje de creación exitosa. En todas las rutas se manejan errores con códigos HTTP adecuados (400 para datos inválidos, 404 si no se encuentra un recurso, 201 para confirmación de creación) y se asegura que solo usuarios autorizados puedan modificar o agregar información, mientras que la entrada de datos siempre se limpia y valida para mantener la seguridad y consistencia de la aplicación.

```
const { Router } = require('express');
const { pool } = require('../db');
const { requireAuth } = require('../middleware/auth');
const { str, isEmail } = require('../util/validators');

const r = Router();

r.get('/users/:registro', async (req,res)=>{
  const [rows] = await pool.query(
    'SELECT registro,nombres,apellidos,email,created_at FROM users WHERE
registro=?',
    [req.params.registro]
  );
  if (!rows.length) return res.status(404).json({ error: 'No existe' });
  res.json(rows[0]);
});
```

```
r.put('/me', requireAuth, async (req,res)=>{
  const nombres = str(req.body.nombres);
  const apellidos = str(req.body.apellidos);
  const email = str(req.body.email).toLowerCase();
  if (!nombres || !apellidos || !isEmail(email)) return res.status(400).json({
error: 'Datos inválidos' });
  await pool.query('UPDATE users SET nombres=?,apellidos=?,email=? WHERE
id=?',[nombres,apellidos,email,req.user.id]);
  res.json({ message: 'Perfil actualizado' });
});

r.get('/courses', async (_req,res)=>{
  const [rows] = await pool.query('SELECT id,code,name FROM courses ORDER BY
name');
  res.json(rows);
});

r.get('/users/:registro/courses', async (req,res)=>{
  const [rows] = await pool.query(
    `SELECT c.id,c.code,c.name FROM approved_courses ac
     JOIN users u ON u.id=ac.user_id
     JOIN courses c ON c.id=ac.course_id
     WHERE u.registro=?`, [req.params.registro]
  );
  res.json(rows);
});

r.post('/me/courses', requireAuth, async (req,res)=>{
  const id = Number(req.body.courseId);
  if (!id) return res.status(400).json({ error: 'courseId requerido' });
  await pool.query('INSERT IGNORE INTO approved_courses (user_id,course_id)
VALUES (?,?)',[req.user.id,id]);
  res.status(201).json({ message: 'Curso agregado' });
});

module.exports = r;
```

Util/validator.js

Este archivo de utilidades define funciones para limpiar y validar datos ingresados por los usuarios. La función str recibe un valor y, si es una cadena, elimina los espacios en blanco al inicio y al final; si no es cadena, devuelve una cadena vacía, asegurando que los datos de texto estén limpios y consistentes. La función isEmail verifica si un valor cumple el formato de correo electrónico válido mediante una expresión regular, retornando true si es correcto o false en caso contrario. Ambas funciones se exportan para ser utilizadas en otras partes del proyecto, garantizando que la información ingresada por los usuarios sea segura y correcta antes de almacenarla o procesarla en la base de datos.

```
const str = s => typeof s === 'string' ? s.trim() : '';
const isEmail = e => /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(e);
module.exports = { str, isEmail };
```

src/db.js

Este archivo configura la conexión a la base de datos MySQL usando mysql2/promise para permitir consultas asíncronas con async/await. Primero, carga las variables de entorno con dotenv para mantener seguros los datos de conexión. Luego, crea un pool de conexiones, definiendo host, usuario, contraseña, nombre de la base de datos y puerto, con un límite de 10 conexiones simultáneas y la opción de esperar si no hay conexiones disponibles, lo que optimiza el rendimiento y la eficiencia en las consultas. Finalmente, exporta el pool para que pueda ser utilizado en otras partes del proyecto, garantizando un acceso seguro y eficiente a la base de datos.

Un pool de conexiones es un conjunto de conexiones ya abiertas hacia la base de datos que la aplicación puede reutilizar cada vez que necesita hacer una consulta. En lugar de abrir y cerrar una nueva conexión cada vez que llega una solicitud (lo cual consume tiempo y recursos), la aplicación toma una conexión disponible del pool, realiza la consulta y luego la devuelve al pool para que otra operación pueda usarla.

```
require('dotenv').config();
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  port: process.env.DB_PORT || 3306,
  waitForConnections: true,
  connectionLimit: 10
});
```

```
module.exports = { pool };
```

src/server.js

El archivo server.js configura y ejecuta el servidor de la API usando Express.

Primero, carga las variables de entorno con dotenv para mantener seguras configuraciones como el puerto y la base de datos.

Luego, importa las dependencias necesarias: express para manejar rutas y peticiones, cors para permitir acceso desde otros dominios, y el pool de MySQL para conectarse a la base de datos. También importa los módulos de rutas para autenticación (authRoutes), publicaciones (postsRoutes) y usuarios/cursos (usersRoutes). Se inicializa la aplicación Express, habilitando CORS y el soporte para recibir datos en JSON. Se define una ruta de health check en /api/health que verifica la conexión con la base de datos y devuelve un estado de funcionamiento. Posteriormente, se registran las rutas de la API con sus respectivos prefijos: /api/auth para autenticación, /api/posts para publicaciones y comentarios, y /api para usuarios y cursos. Finalmente, el servidor se inicia en el puerto definido en la variable de entorno o en 4000 por defecto, mostrando un mensaje en consola indicando que la API está escuchando, asegurando así que el backend esté listo para recibir solicitudes.

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const { pool } = require('./db');

const authRoutes = require('./routes/auth.routes');
const postsRoutes = require('./routes/posts.routes');
const usersRoutes = require('./routes/users.routes');

const app = express();
app.use(cors());
app.use(express.json());

app.get('/api/health', async (req, res) => {
  try { await pool.query('SELECT 1'); res.json({ ok: true }); }
  catch (e) { res.status(500).json({ ok: false, error: e.message }); }
});

app.use('/api/auth', authRoutes);
app.use('/api/posts', postsRoutes);
app.use('/api', usersRoutes);

const port = process.env.PORT || 4000;
```

```
app.listen(port, () => console.log(`API escuchando en :${port}`));
```

src/.env

El archivo. env contiene variables de entorno que configuran el funcionamiento de la aplicación sin exponer información sensible en el código. Define el puerto del servidor (PORT=4000), la clave secreta para JWT (JWT_SECRET) usada en la autenticación, y los datos de conexión a la base de datos MySQL, incluyendo host (DB_HOST), usuario (DB_USER), contraseña (DB_PASSWORD), nombre de la base de datos (DB_NAME) y puerto (DB_PORT), que debe coincidir con el puerto local mapeado en Docker si se usa contenedores. Estas variables permiten que la aplicación sea flexible y segura, facilitando cambios de configuración sin modificar el código fuente.

```
PORT=4000
JWT_SECRET=super_secreto_cambia_esto
DB_HOST=localhost
DB_USER=usac           # o root si prefieres
DB_PASSWORD=usac123     # o root según el usuario que uses
DB_NAME=usac_ratings
DB_PORT=3307            # coincide con el puerto local mapeado en Docker
```

src/init.sql

Este script de base de datos define las tablas y relaciones necesarias para la aplicación. Se crean cinco tablas principales: users para almacenar información de los usuarios con campos como registro, nombres, apellidos, email y contraseña encriptada; courses para los cursos disponibles con código y nombre; posts para las publicaciones de los usuarios, vinculadas a un curso o catedrático y a su autor; comments para comentarios asociados a las publicaciones; y approved_courses que registra los cursos que cada usuario ha aprobado, con claves foráneas hacia users y courses. Además, se incluyen inserciones iniciales (seed) de cursos del área de sistemas, utilizando INSERT ... ON DUPLICATE KEY UPDATE para evitar duplicados y asegurar que cada curso tenga su código y nombre actualizado.

Esto garantiza que la base de datos tenga la estructura completa y los datos esenciales para que la aplicación funcione correctamente, manteniendo relaciones entre usuarios, cursos, publicaciones y comentarios de manera consistente y segura.

```
CREATE TABLE IF NOT EXISTS users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  registro VARCHAR(20) UNIQUE NOT NULL,
  nombres VARCHAR(100) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  email VARCHAR(120) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
```

```sql
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS courses (
  id INT AUTO_INCREMENT PRIMARY KEY,
  code VARCHAR(20),
  name VARCHAR(200) NOT NULL,
  UNIQUE KEY code_unique (code)
);

CREATE TABLE IF NOT EXISTS posts (
  id INT AUTO_INCREMENT PRIMARY KEY,
  author_id INT NOT NULL,
  course_id INT NULL,
  prof_name VARCHAR(150) NULL,
  message TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (author_id) REFERENCES users(id),
  FOREIGN KEY (course_id) REFERENCES courses(id)
);

CREATE TABLE IF NOT EXISTS comments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  post_id INT NOT NULL,
  author_id INT NOT NULL,
  content TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (post_id) REFERENCES posts(id),
  FOREIGN KEY (author_id) REFERENCES users(id)
);

CREATE TABLE IF NOT EXISTS approved_courses (
  user_id INT NOT NULL,
  course_id INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id, course_id),
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (course_id) REFERENCES courses(id)
);

-- Seed de cursos (área de sistemas)
INSERT INTO courses (code, name) VALUES ('ADS1','Análisis y Diseño de Sistemas
1') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('ADS2','Análisis y Diseño de Sistemas
2') ON DUPLICATE KEY UPDATE name=VALUES(name);
```

```sql
INSERT INTO courses (code, name) VALUES ('ACyE1','Arquitectura de Computadoras y Ensambladores 1') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('ACyE2','Arquitectura de Computadoras y Ensambladores 2') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('BD1','Bases de Datos 1') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('BD2','Bases de Datos 2') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('ECON','Economía') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('ED1','Estructura de Datos 1') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('ED2','Estructura de Datos 2') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('GER1','Gerenciales 1') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('GER2','Gerenciales 2') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('IA1','Inteligencia Artificial 1') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('IPC1','Introducción a la Programación y Computación 1') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('IPC2','Introducción a la Programación y Computación 2') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('IAFD','Introducción a los Algoritmos y Flujos de Datos') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('LFP','Lenguajes Formales y de Programación') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('LSIS','Lógica de Sistemas') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('MIA','Manejo e Implementación de Archivos') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('MS1','Modelación y Simulación 1') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('MS2','Modelación y Simulación 2') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('ORGCOMP','Organización Computacional') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('OLC2','Organización de Lenguajes y Compiladores 2') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('PCOM1','Programación Comercial 1') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('PC1','Programación de Computadoras 1') ON DUPLICATE KEY UPDATE name=VALUES(name);
```

```sql
INSERT INTO courses (code, name) VALUES ('PC2','Programación de Computadoras 2')
ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('RED1','Redes de Computadoras 1') ON
DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('RED2','Redes de Computadoras 2') ON
DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('SEM1','Seminario de Sistemas 1') ON
DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('SEM2','Seminario de Sistemas 2') ON
DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('SO1','Sistemas Operativos 1') ON
DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('SO2','Sistemas Operativos 2') ON
DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('SOG1','Sistemas Organizacionales y
Gerenciales 1') ON DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('SWAV','Software Avanzado') ON DUPLICATE
KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('TS1','Teoría de Sistemas 1') ON
DUPLICATE KEY UPDATE name=VALUES(name);
INSERT INTO courses (code, name) VALUES ('TS2','Teoría de Sistemas 2') ON
DUPLICATE KEY UPDATE name=VALUES(name);
```

src/package-lock.json

El archivo package-lock.json no es necesario para la funcionalidad del proyecto, pero se menciona para asegurar la consistencia de las dependencias. Este archivo se genera automáticamente cuando se ejecuta npm install y registra la versión exacta de cada paquete instalado, incluyendo dependencias de las dependencias, garantizando que cualquier desarrollador que instale el proyecto obtenga las mismas versiones de librerías. Aunque no se edita manualmente, es útil para controlar versiones y evitar errores por diferencias entre entornos de desarrollo.

src/package.json

El archivo package.json define la configuración y dependencias del proyecto Node.js. Contiene información general como el nombre del proyecto (name), versión (version) y si es privado (private). Especifica que el proyecto usa CommonJS como sistema de módulos (type: "commonjs") y señala el archivo principal de la aplicación (main: "src/server.js"). En la sección "scripts" se define un comando dev que permite iniciar el servidor con node src/server.js. Además, en "dependencies" se listan los paquetes necesarios para que el proyecto funcione: express para crear el servidor, cors para permitir solicitudes entre dominios, dotenv para manejar variables de entorno,

bcryptjs para encriptar contraseñas, jsonwebtoken para autenticación con JWT y mysql2 para conectarse a la base de datos MySQL.

```
{
  "name": "usac-ratings-backend",
  "version": "1.0.0",
  "private": true,
  "type": "commonjs",
  "main": "src/server.js",
  "scripts": {
    "dev": "node src/server.js"
  },
  "dependencies": {
    "bcryptjs": "2.4.3",
    "cors": "2.8.5",
    "dotenv": "16.4.5",
    "express": "4.19.2",
    "jsonwebtoken": "9.0.2",
    "mysql2": "3.9.7"
  }
}
```

/node_modules

La carpeta node_modules se genera automáticamente al ejecutar comandos como npm i -D @vitejs/plugin-react, donde -D indica que el paquete se instala como dependencia de desarrollo. Esta carpeta almacena todos los paquetes y plugins descargados, en este caso, el plugin de React para Vite, permitiendo que el proyecto los use sin tener que instalarlos manualmente cada vez.

Frontend

Clases Utilizadas

src/components/Filters.css

Este fragmento de Código en CSS para los filtros (especialmente para dropdowns y campos de entrada) se mostrarán en una fila horizontal, con espaciado uniforme entre ellos y todos tendrán el mismo ancho dentro del contenedor. Es un patrón común para formularios de filtrado donde se requiere que todos los campos tengan apariencia consistente.

```
.filters {
  display: flex;
  gap: 12px;
  margin-bottom: 16px;
}

.filters select,
.filters input {
  flex: 1;
}
```

src/components/Filters.js

Este fragmento forma parte del filtro para la calificación por medio de los estudiantes hacia ciertos catedráticos o cursos al igual que permite al usuario encontrar publicaciones relevantes sobre cursos puntuales o catedráticos en particular

```
import './Filters.css';
export default function Filters({filters,setFilters,courses}){
  return <div className="filters">
    <select value={filters.courseId||''} onChange={e=>setFilters(f=>({...f,courseId:e.target.value||undefined}))}>
      <option value="">-- Curso --</option>
      {courses.map(c=><option key={c.id} value={c.id}>{c.name}</option>)}
    </select>
    <input placeholder="Nombre de curso" value={filters.courseName||''}
           onChange={e=>setFilters(f=>({...f,courseName:e.target.value||undefined}))}/>
    <input placeholder="Catedrático" value={filters.profName||''}
           onChange={e=>setFilters(f=>({...f,profName:e.target.value||undefined}))}/>
  </div>;
}
```

src/components/PostCard.css

Este fragmento de código CSS define los estilos para un componente de “tarjeta de publicación” o también en inglés conocida como “post-card” que se utilizará en la sección de publicaciones de la aplicación. Ofrece una presentación visualmente agradable y consistente para la información de cada publicación

```
.post-card {
  background: white;
  border-radius: 8px;
  padding: 16px;
  margin-bottom: 16px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.08);
}

.post-card b {
  font-size: 16px;
  color: #4a90e2;
}

.post-card p {
  margin: 8px 0;
}

.post-card small {
  display: block;
  color: #777;
  margin-bottom: 8px;
}

.post-card input {
  margin-right: 8px;
}
```

src/components/PostCard.js

El propósito de la clase PostCard.js por medio de React es mostrar la publicación individual donde permite a los usuarios añadir comentarios y forma parte fundamental e indispensable en el proyecto, también convierte el formato de fechas ISO a un formato local legible, es la base de la interactividad de los usuarios dentro de la aplicación

```
import React from 'react';
import './PostCard.css';
export default function PostCard({p,onComment}){
  const [text,setText]=React.useState('');
  return <div style={{border:'1px solid #ddd',padding:8,marginBottom:8}}>
    <b>{p.course_name || p.prof_name || 'General'}</b>
    <p>{p.message}</p>
    <small>Autor: {p.registro} · {new Date(p.created_at).toLocaleString()}</small>
    <div>
      <input placeholder="Comentar..." value={text} onChange={e=>setText(e.target.value)} />
      <button onClick={()=>{onComment(p.id,text); setText('');}}>Enviar</button>
    </div>
  </div>;
}
```

src/pages/Auth.css

Este diseño en CSS hace que tenga una interfaz agradable el flujo de autenticación. Sus características son un diseño centrado, esquema de colores que representan y transmiten un sentido de seguridad y confianza; también contienen varios aspectos como focus en los campos para una mejor interacción.

```
.auth-form {
  max-width: 400px;
  margin: 40px auto;
  background: white;
  padding: 24px;
  border-radius: 8px;
  box-shadow: 0 3px 12px rgba(0,0,0,0.1);
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.auth-form h4 {
  color: #4a90e2;
}

.auth-form p {
  color: crimson;
}

.auth-form button {
  margin-top: 8px;
}
```

src/pages/CreatePost.js

El componente de CreatePost es un formulario React que permite a los usuarios crear nuevas publicaciones dentro del programa, precisamente crear nuevas publicaciones en el sistema de valoración de cursos y catedráticos, tiene otras funcionalidades como por ejemplo que obtiene la lista de cursos disponibles desde la API, permite elegir el curso específico, campos de texto para ingresar el nombre del catedrático y hacer comentarios sobre él y por último el envío de la publicación

```
import { useEffect, useState } from 'react';
import { api } from '../api';
export default function CreatePost(){
  const [courses,setCourses]=useState([]); const [courseId,setCourseId]=useState('');
  const [profName,setProfName]=useState(''); const [message,setMessage]=useState('');
  useEffect(()=>{ (async()=>setCourses(await api('/courses')))(); },[]);
  const submit=async(e)=>{ e.preventDefault();
    await api('/posts',{method:'POST',body:JSON.stringify({courseId:courseId||undefined, profName, message})});
    alert('Publicación creada'); window.location.href='/';
  };
  return (<form onSubmit={submit}>
    <h4>Nueva publicación</h4>
    <select value={courseId} onChange={e=>setCourseId(e.target.value)}>
      <option value="">(o elige curso)</option>
      {courses.map(c=><option key={c.id} value={c.id}>{c.name}</option>)}
    </select>
    <input placeholder="o escribe catedrático" value={profName} onChange={e=>setProfName(e.target.value)} />
    <textarea placeholder="Mensaje" value={message} onChange={e=>setMessage(e.target.value)} />
    <button>Publicar</button>
  </form>);
}
```

src/pages/ForgotPassword.js

El componente ForgotPassword permite a los usuarios recuperar y restablecer su contraseña en caso de haberla olvidado, implementando una funcionalidad de “¿Olvidó su Contraseña?” tiene como funcionalidades principales la verificación de identidad mediante su registro académico y correo electrónico, Restablecimiento de contraseña que permite establecer la nueva contraseña del usuario luego de validar su identidad y feedback al usuario que proporciona mensajes informáticos sobre el resultado de las operaciones

```
import { useState } from 'react';
import { api } from '../api';

export default function ForgotPassword(){
  const [registro,setR]=useState(''); const [email,setE]=useState('');
  const [newPassword,setP]=useState(''); const [msg,setMsg]=useState('');
  const submit=async(e)=>{ e.preventDefault(); setMsg('');
    try{ await api('/auth/reset',{method:'POST',body:JSON.stringify({registro,email,newPassword})});
      setMsg('Contraseña actualizada, ya puedes iniciar sesión.');
    }catch(e){ setMsg(e.message); }
  };
  return (<form onSubmit={submit}>
    <h4>Restablecer contraseña</h4>
    <input placeholder="Registro" value={registro} onChange={e=>setR(e.target.value)} />
    <input placeholder="Correo" value={email} onChange={e=>setE(e.target.value)} />
    <input type="password" placeholder="Nueva contraseña" value={newPassword} onChange={e=>setP(e.target.value)} />
    <button>Actualizar</button>
    {msg && <p>{msg}</p>}
  </form>);
}
```

src/pages/Home.css

El componente Home es un contenedor principal del diseño que estructura y organiza el layout de la página de inicio de la aplicación. Sirve como marco visual para presentar el contenido principal de manera organizada y estéticamente coherente al igual que mantiene coherencia visual con el resto de la aplicación mediante la paleta de colores establecida

```
.home-container {
  max-width: 900px;
  margin: 0 auto;
  padding: 16px;
}

.home-container h4 {
  margin-bottom: 16px;
  color: #4a90e2;
}

.home-container a {
  margin-bottom: 16px;
  display: inline-block;
}
```

src/pages/Home.js

El componente Home es la página principal de la aplicación que muestra el listado de publicaciones y permite a los usuarios interactuar con ellas mediante filtros, comentarios y la creación de nuevas publicaciones, su funcionalidad principal es la carga de datos, su sistema de filtrado, gestiones de comentarios y la navegación.

```
1   import { useEffect, useState } from 'react';
2   import { api, getToken } from '../api';
3   import { Link } from 'react-router-dom';
4   import Filters from '../components/Filters.jsx';
5   import PostCard from '../components/PostCard.jsx';
6   import './Home.css';
7
8   export default function Home(){
9     const [courses,setCourses]=useState([]);
10    const [filters,setFilters]=useState({});
11    const [posts,setPosts]=useState([]);
12    const load = async()=>{
13      const qs = new URLSearchParams(filters).toString();
14      setPosts(await api('/posts'+(qs?`?${qs}`:'')));
15    };
16    useEffect(()=>{ (async()=>{ setCourses(await api('/courses')); await load(); })(); },[]);
17    useEffect(()=>{ load(); },[JSON.stringify(filters)]);
18
19    const onComment = async(id,content)=>{
20      if (!getToken()) return alert('Inicia sesión.');
21      await api(`/posts/${id}/comments`,{method:'POST',body:JSON.stringify({content})});
22      await load();
23    };
24
25    return (<div className="home-container">
26      <h4>Publicaciones</h4>
27      <Filters filters={filters} setFilters={setFilters} courses={courses}/>
28      {getToken() && <Link to="/create">Crear publicación</Link>}
29      <div>{posts.map(p=><PostCard key={p.id} p={p} onComment={onComment} />)}</div>
30    </div>);
31  }
32
```

src/pages/Login.js

El componente Login gestiona el proceso de autenticación de usuarios en el sistema, permitiendo el acceso seguro a las funcionalidades de la aplicación mediante la validación de credenciales, tiene como funcionalidades principales la autenticación de usuarios, gestión de tokens de autenticación obtenido tras un inicio de sesión, manejo de errores proporcionando feedback visual ante credenciales

```
import { useState } from 'react';
import { api, setToken } from '../api';
import { useNavigate, Link } from 'react-router-dom';

export default function Login(){
  const [registro,setRegistro]=useState(''); const [password,setPassword]=useState('');
  const [err,setErr]=useState(''); const nav=useNavigate();
  const submit=async(e)=>{ e.preventDefault(); setErr('');
    try{ const {token}=await api('/auth/login',{method:'POST',body:JSON.stringify({registro,password})});
      setToken(token); nav('/');
    }catch(e){ setErr(e.message); }
  };
  return (<form onSubmit={submit}>
    <h4>Iniciar sesión</h4>
    {err && <p style={{color:'crimson'}}>{err}</p>}
    <input placeholder="Registro académico" value={registro} onChange={e=>setRegistro(e.target.value)} />
    <input type="password" placeholder="Contraseña" value={password} onChange={e=>setPassword(e.target.value)} />
    <button>Entrar</button> <Link to="/forgot">¿Olvidaste tu contraseña?</Link>
  </form>);
}
```

src/pages/Profile.js

El componente Profile permite visualizar y gestionar perfiles de usuarios dentro del sistema, mostrando información personal, cursos aprobados y permitiendo la administración de los mismos cuando se accede al perfil propio. Tiene como funcionalidades principales la visualización de perfiles de los distintos usuarios, gestión de cursos aprobados, administración de cursos propios.

```
import { useEffect, useState } from 'react';
import { api, getToken } from '../api';
import { useParams } from 'react-router-dom';

export default function Profile(){
  const { registro } = useParams();
  const [user,setUser]=useState(null);
  const [courses,setCourses]=useState([]); const [mine,setMine]=useState([]);
  const [addId,setAddId]=useState('');

  useEffect(()=>{ (async()=>{
    setUser(await api(`/users/${registro}`));
    setCourses(await api('/courses'));
    setMine(await api(`/users/${registro}/courses`));
  })(); },[registro]);

  const add=async(e)=>{ e.preventDefault(); await api('/me/courses',{method:'POST',body:JSON.stringify({courseId:addId})}); setMine(await api(`/u

  if(!user) return '...';
  return (<div>
    <h4>Perfil de {user.nombres} {user.apellidos}</h4>
    <p>Registro: {user.registro} · Email: {user.email}</p>
    <h5>Cursos aprobados</h5>
    <ul>{mine.map(c=><li key={c.id}>{c.code} - {c.name}</li>)}</ul>
    {getToken() && (
      <form onSubmit={add}>
        <select value={addId} onChange={e=>setAddId(e.target.value)}>
          <option value="">Agregar curso…</option>
          {courses.map(c=><option key={c.id} value={c.id}>{c.name}</option>)}
        </select>
        <button>Agregar</button>
      </form>
    )}
  </div>);
}
```

src/pages/Register.jsx

El propósito de este fragmento es gestionar el proceso de registro de nuevos usuarios en el sistema, permitiendo la creación de cuentas mediante la captura de los datos requeridos por la Universidad de San Carlos, tiene como funcionalidades principales el registro de usuario para crear nuevas cuentas con todos los campos obligatorios, validación de datos al enviar la información al servidor para verificar la información, manejo de errores que proporciona feedback al usuario para la visualización de errores y una interfaz dinámica que genera los campos del formulario.

```
import { useState } from 'react';
import { api } from '../api';
import { useNavigate } from 'react-router-dom';
import './Auth.css';

export default function Register(){
  const [f,setF]=useState({registro:'',nombres:'',apellidos:'',email:'',password:''});
  const [msg,setMsg]=useState(''); const nav=useNavigate();
  const submit=async(e)=>{ e.preventDefault(); setMsg('');
    try{ await api('/auth/register',{method:'POST',body:JSON.stringify(f)}); setMsg('Registrado. Ahora inicia sesión.'); nav('/login'); }
    catch(e){ setMsg(e.message); }
  };
  return (<form className="auth-form" onSubmit={submit}>
    <h4>Registro</h4>
    {msg && <p>{msg}</p>}
    {['registro','nombres','apellidos','email','password'].map(k=>
      <input key={k} placeholder={k} type={k==='password'?'password':'text'}
             value={f[k]} onChange={e=>setF({...f,[k]:e.target.value})} />)}
    <button>Crear cuenta</button>
  </form>);
}
```

src/api.js

Este módulo proporciona una capa de abstracción para las comunicaciones entre el servidor backend, centralizando la gestión de tokens de autenticación, headers HTTP y manejo de errores para todas las solicitudes API. Tiene como funciones principales la administración de los tokens y la autenticación automática

```
const API = import.meta.env.VITE_API_URL || 'http://localhost:4000/api';

export function setToken(t){ localStorage.setItem('tk', t); }
export function getToken(){ return localStorage.getItem('tk'); }
export async function api(path, opts={}){
  const headers = { 'Content-Type':'application/json', ...(opts.headers||{}) };
  const tk = getToken(); if (tk) headers.Authorization = 'Bearer '+tk;
  const res = await fetch(`${API}${path}`, { ...opts, headers });
  if (!res.ok) {
    let msg = 'Error';
    try { msg = (await res.json()).error || msg; } catch {}
    throw new Error(msg);
  }
  return res.json();
}

```

src/app.js

El componente App sirve como el layout principal y contenedor de la aplicación, proporcionando la estructura base, navegación consistente y manejo de autenticación en toda la aplicación. Tiene como funcionalidad principal ser el Layout base y establece una estructura visual principal de la aplicación, proporciona menú de navegación accesible desde todas las páginas, gestiona la autenticación mostrando diferentes opciones para los usuarios autenticados y no autenticados

```
import { Link, Outlet, useNavigate } from 'react-router-dom';
import React from 'react';
import { getToken } from './api';

export default function App(){
  const nav = useNavigate();
  return (
    <div style={{maxWidth:900, margin:'0 auto', padding:16}}>
      <header style={{display:'flex', gap:12, alignItems:'center', marginBottom:12}}>
        <h3>USAC Ratings</h3>
        <Link to="/">Inicio</Link>
        {getToken() ? <>
          <Link to="/create">Nueva publicación</Link>
          <button onClick={()=>{ localStorage.clear(); nav('/'); }}>Salir</button>
        </> : <>
          <Link to="/login">Entrar</Link>
          <Link to="/register">Registro</Link>
        </>}
      </header>
      <Outlet/>
    </div>
  );
}
```

src/index.css

Este archivo CSS establece los estilos base y el sistema de diseño consistente para toda la aplicación, definiendo la apariencia visual unificada y las reglas de estilo fundamentales. Tiene como secciones principales el reset básico y normalización es decir que elimina imágenes y paddings por defecto de todos los elementos y garantixa un punto de partido para todos los elementos html. Tiene estilos base de tipografía y color ya que define una familia de fuentes principales y un color de fondo general

```css
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Inter', Arial, sans-serif;
  background-color: #f4f6f8;
  color: #333;
}

a {
  text-decoration: none;
  color: #4a90e2;
}

a:hover {
  text-decoration: underline;
}

button {
  cursor: pointer;
  padding: 8px 16px;
  border: none;
  border-radius: 6px;
  background-color: #4a90e2;
  color: white;
  font-weight: 500;
  transition: background 0.3s;
}

```

```css
button:hover {
  background-color: #357ab8;
}

input, select, textarea {
  padding: 8px 12px;
  border: 1px solid #ccc;
  border-radius: 6px;
  font-size: 14px;
  width: 100%;
}

header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 16px 0;
  border-bottom: 1px solid #ddd;
  margin-bottom: 24px;
}

header h3 {
  color: #4a90e2;
}

form {
  background: white;
  padding: 24px;
  border-radius: 8px;
  box-shadow: 0 3px 8px rgba(0,0,0,0.08);
  max-width: 500px;
  margin: 0 auto 24px;
  display: flex;
  flex-direction: column;
  gap: 12px;
}
```

src/main.jsx

Este archivo sirve como el punto de entrada principal de la aplicación React, configurando el enrutamiento, inicializando la aplicación y montando todo el árbol de componentes en el DOM. Tiene como funciones principales la inicialización de React configura el root de React DOM para renderizar la aplicación, Configura el enrutamiento al definir la estructura de rutas de la aplicación, Integración de componentes ya que importa y organiza todos los componentes de la página y por último el manejo de rutas no definidas ya que redirige rutas inexistentes a la página principal

```jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import App from './App.jsx';
import Login from './pages/Login.jsx';
import Register from './pages/Register.jsx';
import ForgotPassword from './pages/ForgotPassword.jsx';
import Home from './pages/Home.jsx';
import CreatePost from './pages/CreatePost.jsx';
import Profile from './pages/Profile.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <Routes>
      <Route path="/" element={<App />}>
        <Route index element={<Home />} />
        <Route path="login" element={<Login />} />
        <Route path="register" element={<Register />} />
        <Route path="forgot" element={<ForgotPassword />} />
        <Route path="create" element={<CreatePost />} />
        <Route path="profile/:registro" element={<Profile />} />
        <Route path="*" element={<Navigate to="/" />} />
      </Route>
    </Routes>
  </BrowserRouter>
);

```

/package.json

Este archivo define la configuración fundamental del proyecto fronted, incluyendo metadatos, dependencias y scripts de ejecución. Sirve como el manifiesto principal que gestiona la construcción y desarrollo de la aplicación

```json
{
  "name": "usac-ratings-frontend",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  ▷Debug
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.26.2"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^5.0.2",
    "vite": "^7.1.4"
  }
}

```

/vite.config.json

Este archivo configura el entorno de desarrollo y construcción de la aplicación utilizando Vite como herramienta de build. Define cómo se procesa, compila y optimiza el código para desarrollo y producción.

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
});
```

/package-lock.json

Este archivo garantiza instalaciones consistentes y reproducibles de las dependencias del proyecto. Bloquea las versiones exactas de todos los paquetes y sus dependencias transitivas, asegurando que todos los desarrolladores y entornos de despliegue utilicen exactamente las mismas versiones de cada paquete.

/node_modules

La carpeta node_modules se genera automáticamente al ejecutar comandos como npm i -D @vitejs/plugin-react, donde -D indica que el paquete se instala como dependencia de desarrollo. Esta carpeta almacena todos los paquetes y plugins descargados, en este caso, el plugin de React para Vite, permitiendo que el proyecto los use sin tener que instalarlos manualmente cada vez.

Serie de pasos para correr la app

1. Crear y correr el contenedor de MySQL

- Se descargará automáticamente la imagen si no la tienes

- Puedes cambiar el puerto si 3307 ya está en uso

docker run --name usac-mysql -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=usac_ratings -p 3307:3306 -d mysql:8

1. Crear y correr el contenedor de MySQL

- Se descargará automáticamente la imagen si no la tienes

- Puedes cambiar el puerto si 3307 ya está en uso

docker run --name usac-mysql -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=usac_ratings -p 3307:3306 -d mysql:8

2. Verificar que el contenedor está corriendo

docker ps

Deberías ver algo como:

CONTAINER ID IMAGE COMMAND PORTS NAMES

aa036176138a mysql:8 "docker-entrypoint.s…" 0.0.0.0:3307->3306/tcp usac-mysql

3. (Opcional) Crear un nuevo usuario 'usac' con contraseña 'usac123'

 # - Si quieres usar root, puedes omitir este paso

docker exec -i usac-mysql mysql -uroot -proot -e "CREATE USER IF NOT EXISTS 'usac'@'%' IDENTIFIED BY 'usac123'; GRANT ALL PRIVILEGES ON usac_ratings.* TO 'usac'@'%'; FLUSH PRIVILEGES;"

4. Ubicarse en la carpeta donde está el archivo init.sql

cd "C:\Users\Alex\Desktop\TAREAS\Tareitas University\TALLER-3\backend"

5. Cargar el esquema init.sql en la base de datos

- Si creaste el usuario 'usac', reemplaza -uroot -proot por -uusac -pusac123

type init.sql | docker exec -i usac-mysql mysql -uroot -proot usac_ratings

 # 6. Verificar que se crearon las tablas y el seed de cursos

- Cambia las credenciales si usas el usuario 'usac'

docker exec -it usac-mysql mysql -uroot -proot usac_ratings -e "SHOW TABLES; SELECT COUNT(*) AS cursos FROM courses;"

 # 7. Copiar o renombrar el archivo de ejemplo de variables de entorno

- Cambia .env.example a .env para que Node.js lo lea

copy .env.example .env

o en PowerShell

rename-item .env.example .env

 # 8. Revisar y ajustar las variables en .env

- Si creaste el usuario 'usac', usa:

DB_USER=usac

DB_PASSWORD=usac123

DB_PORT=3307 # el puerto que mapeaste en Docker

- Si prefieres usar root:

DB_USER=root

DB_PASSWORD=root

DB_PORT=3307

 # 9. Instalar dependencias del backend

npm install

10. Levantar el backend

npm run dev

Deberías ver algo como:

API escuchando en :4000

 # 11. Probar la ruta de salud (health)

- Abrir navegador o usar Postman/Curl

- URL: http://localhost:4000/health

- Deberías recibir:

{"ok":true}

 # 12. Levantar el frontend

cd "../frontend"

npm install

Instalar plugin de React para Vite (si no está)

npm i -D @vitejs/plugin-react

 # 13. Configurar Vite para React

- Abrir vite.config.js y agregar:

import { defineConfig } from 'vite';

import react from '@vitejs/plugin-react';

export default defineConfig({ plugins: [react()] });

 # 14. Levantar el frontend

npm run dev

Abrir en navegador:

http://localhost:5173/

Deberías ver tu app corriendo sin errores de "React is not defined

Conclusiones

- Se ha establecido una arquitectura cliente-servidor robusta utilizando React para el frontend.
- La estructura de datos está bien organizada y es mantenible
- El proyecto mantiene un código limpio y bien estructurado, siguiendo mejores prácticas de desarrollo en React. Los componentes son modulares y reutilizables, facilitando el mantenimiento y escalabilidad futura del sistema.
- La estructura del proyecto está organizada de manera lógica, con una clara separación entre componentes, utilidades y estilos, lo que facilitará futuras actualizaciones y correcciones.

Recomendaciones

Recomendamos leer el manual técnico en el orden presentado, comenzando por la introducción y componentes específicos. Esto proporcionará el contexto necesario para comprender completamente e desarrollo del sistema.